StarForth VM and Physics-Driven Adaptive Runtime

Volume I of the StarshipOS Technical Reference

Robert A. James

Contents

Chapter 1

Introduction

1.1 What StarForth Is

1.2 Technology Stack

Table 1.1: StarshipOS technology stack

Layer	Component
Future OS	StarshipOS (self-hosting)
Kernel	LithosAnanke v1.5.3 / StarKernel (bare-metal UEFI)
VM	StarForth v3.1.0 (FORTH-79, physics-driven adaptive RT)
Host platform	Linux, L4Re/Fiasco.OC, bare metal (amd64, aarch64, riscv64)

1.3 Thermodynamic Modeling Language

1.4 Document Scope and Organization

1.5 Experimental Campaign Notes

1.6 StarForth, LithosAnanke, and StarshipOS: A Technical Overview

Author: Robert A. James. Date: 9 April 2026. Status: Working Document.

1.6.1 StarForth

What It Is

StarForth is a pure C99 FORTH-79 virtual machine with no libc dependencies. It is self-adaptive: it observes its own execution behavior and continuously adjusts internal parameters to maintain a conservation invariant called K≡1.0. It runs on x86_64, aarch64, and RISC-V.

Why FORTH

FORTH is correct for this work for reasons beyond familiarity. FORTH words are discrete, classifiable, observable computational units — each word has a definite stack-effect signature, each word consumes a known quantity of computational energy, each word is a particle with measurable properties. This is the foundation of StarForth's self-adaptive behavior.

The Primitive Taxonomy

StarForth implements 23 word modules (18 FORTH-79 standard, 5 StarForth extensions). Every primitive word carries quantum-analog properties derived from its stack-effect signature:

Spin. Derived from net stack-effect arithmetic. A word consuming two values and producing one has spin $-\frac{1}{2}$; these values fall directly from `( n n - n )` notation, as half-integer spin falls from the Dirac equation.

Charge. The net stack delta. Charge conservation across a complete program means the program is stack-balanced.

Heat. Execution frequency and recency. Heat decays over time, analogous to radioactive decay.

Mass. Bytecode footprint of the word definition.

Color. Binding state: white words are fully resolved; colored words carry unresolved forward references.

Three of these quantities — spin, charge, and color — behave as genuine conservation laws across well-formed programs. This is a structural property of the system discovered empirically and subsequently formalized in Isabelle/HOL.

The SSM and James Law

The SSM (Steady State Machine) is the self-adaptive core of StarForth. It monitors execution behavior through instrumented subsystems and continuously adjusts internal parameters to maintain K≡1.0.

James Law ($K \equiv 1.0$) is a conservation invariant: the normalized total execution energy of the StarForth universe is conserved at unity across all configurations and workloads. Validation basis: 38,400 experimental runs spanning 128 factorial parameter configurations, CV below 1%, published on SSRN (abstract 5930134).

The SSM operates through four mechanisms:

Rolling Window of Truth (RWOT). A sliding observation window encoding five observables: word identity, prev→word transition, instruction pointer position, execution heat, and causal stack depth. Updated at heartbeat tick intervals, making it a coarse-grained lattice observable.

Adaptive Heartbeat. A self-adjusting timer governing when the SSM evaluates system state and adjusts parameters. The system's intrinsic rhythm.

Physics Hooks. Every word execution in the inner interpreter loop is bracketed by `physics_pre_execute` and `physics_post_execute` calls, updating heat, decay, and window measurements.

Decay. Execution heat decays between heartbeat ticks, keeping the observable window relevant to current rather than historical behavior.

The Phase Space and Attractor

Six workload types have been studied experimentally: STABLE, VOLATILE, TRANSITION, TEMPORAL, DIVERSE, and OMNI. Each produces a distinct trajectory in phase space. All trajectories converge to the same attractor: K≡1.0. Phase-space portrait visualizations confirm attractor convergence across all workload types on all three target architectures.

The Manifold Navigation Controller (Theoretical)

Given a bounded system with a known conservation invariant and predictable attractor manifold, a controller can:

1. Identify the system's current phase-space position using three observables: RWOT (position), adaptive heartbeat metadata (velocity), and spin-class transition rate (acceleration — requires instrumentation).
2. Compute a perturbance vector sufficient to transport the system from its current manifold to an arbitrary target manifold.
3. Inject the perturbance — the burn — and release.
4. Allow the system to coast back to K≡1.0 under its own restoring force.

The controller fires once and releases. The conservation invariant guarantees the return journey. This orbital-mechanics model scales from the word level through the VM, OS, FPGA, and prospectively to custom silicon.

—

1.6.2 LithosAnanke

What It Is

LithosAnanke is a bare-metal operating system for x86_64, currently at milestone M7 (VM integration). It boots via UEFI, manages physical and virtual memory, handles interrupts via IDT/APIC, maintains a timer, manages a heap, and hosts the StarForth VM as its primary computational substrate. Written in C99 with no external dependencies.

The Capsule System

The primary organizational unit is the *capsule*: a content-addressed, 64-byte cache-line-aligned descriptor. A capsule's ID is its content hash. Identity is content: two capsules with identical content have identical IDs. Modification produces a new capsule with a new ID. Capsules are immutable by design; mutation produces new capsules.

Component Architecture

Hera. The foundational VM and heartbeat. The system's ground state.

Hermes. Message entropy and TTL decay. Manages the message-passing fabric, monitors message lifecycle, and reaps expired messages. The natural home of the perturbance controller extension.

Artemis. Persistence and event sourcing. Manages the capsule store, maintains the event log, and handles durable state.

Components communicate exclusively through message passing. There is no shared mutable state between components.

Authentication and Identity

LithosAnanke implements a hardware-rooted authentication model with no usernames and no passwords. Identity is carried on a physical thumb drive containing two raw block partitions.

Access control is governed by temporal ACL grants that decay via the same Hermes TTL mechanism that governs message lifetime. The default state of the permission system is no access. Decay to zero requires no explicit revocation.

Normalized Virtual Block Space

From any VM's perspective, LithosAnanke presents a single contiguous block address space. Physical origin — local drive, system hardware, or remote cloud mount — is transparent. Blocks appear as grants are issued; blocks vanish as grants decay. Security is expressed as address-space geometry.

The Three UX Tiers

CLI. The traditional FORTH terminal interface. Full power, zero overhead. The REPL is operational.

GUI. A wireframe soundstage rendered from rasterized vector graphics. Words are physical objects; the user assembles colon definitions spatially. The FORTH text is always visible underneath.

VR. The full holodeck realization. The user stands inside the phase space. Words have mass, spin, heat, and charge as observable properties. Manifold navigation is literally navigable space.

All three tiers are windows into identical underlying mechanics.
—

1.6.3 StarshipOS

The SSPU

The SSPU (Steady State Processing Unit) is a novel processor architecture extending the StarForth/SSM work into hardware. It communicates entirely via a message-passing fabric called Gefyra. There are no traditional buses. Targeted for initial FPGA implementation on a Digilent Genesys ZU (Zynq UltraScale+).

Milestone Status

Milestone	Description	Status
M0	UEFI boot	Complete
M1	Physical memory manager	Complete
M2	Virtual memory manager	Complete
M3	IDT/APIC interrupt handling	Complete
M4	Timer	Complete
M5	Heap	Complete
M6	–	Complete
M7	VM integration	Active frontier
–	Manifold Navigation Controller	Theoretical
–	SSPU FPGA implementation	Planned (Genesys ZU)

Design Philosophy

The recurring pattern in this architecture is the elimination of problem domains rather than their simplification. No scheduler, no traditional memory manager, no username/password authentication, no logout ceremony, no coordination layer for distributed state — each elimination is a deliberate architectural decision. The question at each design point was not "how do we implement this?" but "do we need this at all?"

Key Terms

Term	Definition
K≡1.0	James Law — conservation invariant of normalized execution energy
SSM	Steady State Machine — self-adaptive core of StarForth
RWOT	Rolling Window of Truth — coarse-grained phase-space observable
SSPU	Steady State Processing Unit — novel message-passing processor
Gefyra	Message-passing fabric connecting SSPU processing elements
Capsule	Content-addressed, 64-byte aligned immutable descriptor
Hera	Foundational VM and heartbeat component
Hermes	Message entropy, TTL decay, lifecycle management
Artemis	Persistence and event sourcing

Chapter 2

FORTH-79 Interpreter

2.1 Memory Model

2.2 Dictionary Structure

2.3 FORTH-79 Word Categories

Table 2.1: StarForth word implementation files

File	Scope
`arithmetic_words.c`	`+ - * / MOD ABS MIN MAX`
`stack_words.c`	`DUP DROP SWAP ROT OVER NIP TUCK`
`control_words.c`	`IF ELSE THEN DO LOOP BEGIN UNTIL WHILE`
`defining_words.c`	`: ; CREATE DOES> VARIABLE CONSTANT`
`memory_words.c`	`@ ! C@ C! MOVE FILL`
`return_stack_words.c`	`>R R> R@ RDROP 2>R 2R@ 2R>`
`double_words.c`	`2DUP 2DROP 2SWAP 2@ 2! D+ D-`
`logical_words.c`	`AND OR XOR NOT INVERT LSHIFT RSHIFT`
`io_words.c`	`EMIT KEY TYPE CR TAB SPACE ACCEPT`
`string_words.c`	`S" SLITERAL` and string operations
`block_words.c`	`BLOCK BUFFER LOAD THRU FLUSH`
`format_words.c`	`.( .R .S HEX DECIMAL BASE`
`system_words.c`	`BYE ABORT INCLUDE STATE`
`dictionary_words.c`	`FIND SEARCH-WORDLIST WORDS`
`vocabulary_words.c`	`VOCABULARY DEFINITIONS FORTH-WORDLIST`
`q48_16_words.c`	$\mathbb{Q}_{48.16}$ fixed-point word definitions
`starforth_words.c`	StarForth-specific extensions

2.4 Initialization and Boot

2.5 Q48.16 Fixed-Point Arithmetic

Chapter 3

Physics-Driven Adaptive Runtime

StarForth's adaptive runtime models its own execution behavior using execution statistics as a proxy for thermodynamic quantities. Using execution frequency as a proxy for thermal energy, the runtime tracks *heat* per word, allows heat to *decay* over time, and reasons about *stability* in statistical terms. These are descriptive tools borrowed from thermodynamics as a modeling convenience; they are not claims of physical thermodynamics. The canonical term definitions are given in `ONTOLOGY.md`.

The runtime is governed by seven configurable feedback loops and an overarching Steady-State Machine (SSM). Each loop pairs a positive (accumulation) signal that drives the runtime toward optimization with a negative (stabilization) signal that prevents runaway growth and holds the system near equilibrium. All seven loops are independently togglable at build time, enabling factorial Design of Experiments (DoE) campaigns that measure each loop's contribution in isolation.

3.1 Overview: Seven Feedback Loops

Loop	Name	Accumulation signal	Stabilization signal
#1	Execution Heat	word execution increments heat	Loop #3 decay; cold words demoted
#2	Rolling Window	every execution recorded to buffer	wrap overwrite; adaptive shrinking
#3	Linear Decay	slope starts at 1/3	decay reduces heat; slope adjusts ±5%
#4	Pipelining	word→word transitions recorded	counts age out; miss-rate feedback
#5	Window Inference	window grows with diversity	Levene's test shrinks to minimum
#6	Decay Inference	regression extracts decay rate	ANOVA early-exit; fit-quality bound
#7	Adaptive Heartrate	tick counter increments	inference cadence falls when stable

Table 3.1: The seven feedback loops, with their accumulation and stabilization mechanisms.

Any loop can be disabled at build time by setting its flag to zero; setting all seven to zero yields the baseline configuration used as the DoE reference point. A global configuration table is given in Section 3.11.

3.2 Loop #1 — Execution Heat

Loop #1 counts word usage to identify hot execution paths. The implementation resides in `src/dictionary_heat_optimization.c`. Every word execution increments the `execution_heat` field in the word's `DictEntry`, using execution frequency as a proxy for thermal energy. Words whose heat crosses the promotion threshold migrate into the `hotwords_cache` for constant-time lookup; words falling below `HEAT_CACHE_DEMOTION_THRESHOLD` (default 10) are evicted from the cache. The `PHYSICS-RESET-STATS` word resets all heat counters.

Two special flags modify decay behavior for individual words. `WORD_FROZEN` words never lose heat—their `execution_heat` does not participate in the Loop #3 decay sweep at all. `WORD_PINNED` words decay normally but are prevented from reaching zero: their heat is held at a minimum of one after any decay step. These flags allow critical words to remain permanently warm without requiring the heat tracking mechanism to be suppressed globally.

Loop #1 interacts with Loop #3 (Linear Decay), which provides the stabilization signal that prevents unlimited heat accumulation, and with the hot-words cache (Section 3.13), which consumes Loop #1's output. The build flag is `ENABLE_LOOP_1_HEAT_TRACKING`. Formal properties of the heat lifecycle are established in `proof/StarForth_Loop1_Heat.thy`, which proves heat non-negativity, strict monotonicity of increment, an upper bound of `MAX_HEAT`, immunity to decay for `WORD_FROZEN` words, and a heat floor of 1 for `WORD_PINNED` words after any decay step.

`HEAT_CACHE_DEMOTION_THRESHOLD` Heat value below which a word is evicted from the hot-words cache. Default: 10.

`ENABLE_LOOP_1_HEAT_TRACKING` Build flag; set to 0 to disable the entire heat-tracking subsystem.

`WORD_FROZEN` Word flag: heat does not decay.

`WORD_PINNED` Word flag: heat cannot decay to zero.

3.3 Loop #2 — Rolling Window of Truth

Loop #2 captures the execution sequence in a circular buffer to provide representative data for the inference engine. The implementation resides in `src/rolling_window_of_truth.c`. The key data structure is `RollingWindowOfTruth`, which is embedded in the `VM` struct and provides a circular buffer of recently executed word identifiers, double-buffered snapshots for safe concurrent access by the heartbeat thread, and an adaptive effective window size field controlled by Loops #2 and #5.

The window grows to `ROLLING_WINDOW_SIZE` (default 4096) before wrapping and becomes "warm" after 1024 executions. Only a warm window holds representative data; inference operations check the `is_warm` flag before running. Stabilization is twofold: oldest entries are overwritten on wrap, and adaptive shrinking reduces `effective_window_size` whenever pattern-diversity growth falls below `ADAPTIVE_GROWTH_THRESHOLD` (1%). The diversity check runs every `ADAPTIVE_CHECK_FREQUENCY` (256) executions, retains `ADAPTIVE_SHRINK_RATE` (75%) of the current window per cycle, and never shrinks below `ADAPTIVE_MIN_WINDOW_SIZE` (256).

Loop #2 interacts with Loop #5, which applies Levene's test to determine whether the window has reached its minimum sufficient size for the current workload. The build flag is `ENABLE_LOOP_2_ROLLING_WINDOW`. Formal properties are established in `proof/StarForth_Loop2_Window.thy`, which models both the effective window size and the actual (fill-level) window size and proves that the effective window remains within the [`ADAPTIVE_MIN`, `ROLLING_WINDOW_SIZE`] bounds.

`ROLLING_WINDOW_SIZE` Maximum window size and initial capacity. Default: 4096.

`ADAPTIVE_CHECK_FREQUENCY` Executions between diversity checks. Default: 256.

`ADAPTIVE_GROWTH_THRESHOLD` Pattern-diversity growth rate (%) below which shrinking triggers. Default: 1%.

`ADAPTIVE_SHRINK_RATE` Fraction of window retained per shrink cycle. Default: 75%.

`ADAPTIVE_MIN_WINDOW_SIZE` Floor for the effective window size. Default: 256.

3.4 Loop #3 — Linear Decay

Loop #3 ages words to prevent unbounded heat accumulation; it is primarily a negative-feedback loop. It does not have a separate implementation file but is part of the heat-tracking subsystem in `src/dictionary_heat_optimization.c`, driven by the heartbeat dispatcher in `src/vm.c`. On each decay sweep, every word's heat is reduced according to:

$$H(t) = \max\!\big(0,\ H_0 - d \cdot \Delta t\big), \tag{3.1}$$

where d is the decay constant expressed in $\mathbb{Q}_{48.16}$ fixed point as `DECAY_RATE_PER_US_Q16`, giving a half-life of roughly six to seven seconds for a word carrying 100 heat units. A minimum interval of `DECAY_MIN_INTERVAL` (1 μs) guards against over-decay when the heartbeat fires in rapid succession.

The decay slope is stored as a $\mathbb{Q}_{48.16}$ value (`decay_slope_q48`, initialized to 1/3) and self-adjusts by $\pm 5\%$ against the hot-to-stale word ratio observed during each heartbeat sweep: an excess of hot words increases the slope (faster decay), while an excess of stale words decreases it (slower decay). This self-adjustment is the accumulation signal for a primarily stabilizing loop. Loop #6 provides the longer-horizon inference mechanism that recalculates the decay slope from exponential regression over the heat trajectory.

The build flag is `ENABLE_LOOP_3_LINEAR_DECAY`. Formal properties are established in `proof/StarForth_`
which proves that the decay operation is non-negative, monotonically decreasing, and that `WORD_FROZEN` words are immune to decay steps.

`DECAY_RATE_PER_US_Q16` Heat decay per microsecond in $\mathbb{Q}_{48.16}$ fixed point. Default: 1/65536 heat per μs.

`DECAY_MIN_INTERVAL` Minimum elapsed time before a decay sweep fires. Default: 1 μs.

`decay_slope_q48` Per-VM $\mathbb{Q}_{48.16}$ value holding the current decay slope; initialized to 1/3; adjusted $\pm 5\%$ by Loop #3's self-tuning and updated by Loop #6.

3.5 Loop #4 — Word Transition Prediction (Pipelining)

Loop #4 tracks word-to-word transitions to enable speculative prefetch, reducing dictionary lookup latency by starting the lookup of word B during the execution of word A. The implementation resides in `src/physics_pipelining_metrics.c`. Each `DictEntry` carries a `WordTransitionMetrics` substructure that records transition counts toward each successor, a total-transitions counter, and $\mathbb{Q}_{48.16}$ latency accounts for prefetch savings and misprediction cost. Global aggregate state is held in `PipelineGlobalMetrics`, which includes `prefetch_attempts`, `prefetch_hits`, `suggested_next_size` (the binary-chop window candidate), and `last_checked_accuracy`. Transition probability is computed in $\mathbb{Q}_{48.16}$ fixed point to avoid floating-point arithmetic on the hot path:

$$P(\text{from} \to \text{to}) = \frac{\texttt{transition\_heat}[\text{to}]}{\texttt{total\_transitions}}. \tag{3.2}$$

The design proceeds in phases. Instrumentation (recording transitions) is implemented and active. Prediction analysis and the actual speculative prefetch are subsequent phases. Transition counts age out over time, and a miss-rate signal drives binary-chop search over `TRANSITION_WINDOW_SIZE` through `PipelineGlobalMetrics.suggested_next_size`. The build flag is `ENABLE_LOOP_4_PIPELINING_METRICS`. Formal safety properties are established in `proof/StarForth_`

`TRANSITION_WINDOW_SIZE` Context depth (number of prior words) used for transition prediction. Default: 2.

`SPECULATION_THRESHOLD` Minimum transition probability required before a speculative prefetch is issued. Default: 0.50.

`MIN_SAMPLES` Minimum recorded transitions before the predictor begins speculating. Default: 10.

`MISPREDICTION_COST` Penalty charged for a wrong prediction, in nanoseconds. Default: 25 ns.

3.6 Loop #5 — Window Width Inference

Loop #5 finds the optimal rolling-window size through statistical testing, using the inference engine in `src/inference_engine.c`. The effective window size expands toward `ROLLING_WINDOW_SIZE` when pattern diversity rises above threshold. For stabilization, it applies Levene's test for equality of variance: the heat trajectory is split into K disjoint chunks, each chunk's variance is computed independently, and the Levene test statistic W is compared against a critical value ($W \approx 6.5$ at $\alpha = 0.05$). When $W \leq$ critical, the variances are stable and the window has reached its minimum sufficient size, preventing over-capture of redundant pattern data.

The binary-chop mechanism (`loop_5_binary_chop_suggest_window()`) is invoked by the heartbeat window-tuner plugin (`vm_tick_window_tuner()`, which runs at `WINDOW_TUNING_FREQUENCY` ticks). It computes the current prefetch accuracy and proposes a new effective window size, which the window tuner applies if it differs from the current size.

The Levene test's statistical correctness—that it correctly detects non-uniform execution diversity—is taken as an explicit named axiom (`inference_window_clamped_valid`) in `proof/StarForth_Loop5_` carrying a documented human-audit obligation against `src/inference_engine.c`. All other properties (boundedness, monotonicity, ANOVA early-exit) are fully mechanised.

`WINDOW_TUNING_FREQUENCY` Heartbeat ticks between window-tuner invocations. Default: 1000.

`ADAPTIVE_MIN_WINDOW_SIZE` Hard floor on the effective window size. Default: 256.

3.7 Loop #6 — Decay Slope Inference

Loop #6 extracts the optimal decay rate by exponential regression over the heat trajectory, using the inference engine in `src/inference_engine.c`. Fitting the log-linear model

$$\ln(\text{heat}[t]) = \ln(h_0) - \text{slope} \cdot t \tag{3.3}$$

yields a closed-form `adaptive_decay_slope`. An ANOVA early-exit guards cost: if `has_variance_stabilized` reports that variance changed by less than 5% since the last inference, the computation is skipped and the cached result reused, saving an estimated 5,000–10,000 CPU cycles per skipped run. The slope is bounded by fit quality (`slope_fit_quality_q48`), and `inference_outputs_validate()` rejects slopes that are zero or exceed 100.

The build flag is `ENABLE_LOOP_6_DECAY_INFERENCE`. Unlike Loop #5, Loop #6 requires no statistical axiom: all invariants are proved purely from the clamping of the raw ordinary least-squares slope to [`DECAY_SLOPE_MIN`, `DECAY_SLOPE_MAX`] before writing to the VM state. This is established in `proof/StarForth_Loop6_DecayInf.thy`.

`slope_fit_quality_q48` $\mathbb{Q}_{48.16}$ quality metric for the current regression fit; used to gate slope updates.

`HEARTBEAT_INFERENCE_FREQUENCY` Ticks between full inference runs. Default: 5000.

`SLOPE_VALIDATION_FREQUENCY` Ticks between decay-slope validation passes. Default: 5000.

3.8 Loop #7 — Adaptive Heartrate

Loop #7 adjusts tick frequency to balance responsiveness against inference cost. All time-driven VM operations are coordinated through a single dispatcher, `vm_tick()`, in `src/vm.c`. The `HeartbeatState` struct, embedded in the `VM` struct and declared in `include/vm.h`, holds the tick counter and per-plugin cursors:

```
typedef struct {
    uint64_t tick_count;             /* total ticks since VM init */
    uint64_t last_window_tune_tick; /* tick of last window tune */
    uint64_t last_slope_tune_tick;  /* tick of last decay validation
        */
    int      heartbeat_enabled;      /* 1 = active, 0 = disabled */
} HeartbeatState;
```

The main interpreter increments an execution counter and fires `vm_tick()` every `HEARTBEAT_FREQUENCY` (default 256) word executions. The dispatcher increments `tick_count` and invokes each registered plugin at its configured interval: the window-tuner plugin (`vm_tick_window_tuner()`, every `WINDOW_TUNING_FREQUENCY` ticks) and the slope-validator plugin (`vm_tick_slope_validator()`, every `SLOPE_VALIDATION_FREQUENCY` ticks).

When the system is stable, full inference runs less often, reducing CPU overhead. The background `HeartbeatWorker` thread can be enabled with `HEARTBEAT_THREAD_ENABLED=1`, in which case it wakes at `HEARTBEAT_THREAD_INTERVAL_MS` and calls `vm_tick()` independently of the execution hot-path. In threaded mode, state shared between the execution thread and the heartbeat thread is protected by `vm->tuning_lock`.

Implementation status: `HeartbeatTickSnapshot` and `tick_buffer` are declared in `include/vm.h`, but `heartbeat_export_csv()` is not yet implemented. The instrumentation plan is recorded in `docs/03-architecture/heartbeat-system/instrumentation-plan.md`.

The build flag is `ENABLE_LOOP_7_ADAPTIVE_HEARTRATE`. Formal invariants are established in `proof/StarForth_Loop7_Heartrate.thy`, which proves that `tick_target_ns` is always greater than zero, remains within [`TICK_MIN_NS`, `TICK_MAX_NS`], and that `hb_tick_count` increases monotonically.

`HEARTBEAT_FREQUENCY` Word executions between ticks. Default: 256.

`WINDOW_TUNING_FREQUENCY` Ticks between window-tuner calls. Default: 1000.

`SLOPE_VALIDATION_FREQUENCY` Ticks between decay-slope validation. Default: 5000.

`HEARTBEAT_THREAD_ENABLED` Build flag; 1 enables the background POSIX thread. Default: 0 (synchronous mode).

`HEARTBEAT_THREAD_INTERVAL_MS` Wake interval for the background thread when enabled. Default: 100 ms.

3.9 L8 Jacquard Mode Selector

The L8 Jacquard Mode Selector is a meta-coordinator that sits above the seven feedback loops. It is implemented in `src/ssm_jacquard.c` and its current mode is held in `vm->ssm_l8_state`. Rather than a feedback loop in the same sense as Loops #1–#7, L8 is a Steady-State Machine (SSM) that switches the runtime between predefined operating configurations based on observed attractor-bucket statistics aggregated by the heartbeat mechanism.

Using execution statistics as a proxy for thermodynamic quantities, the SSM tracks execution heat, entropy (workload diversity), pipeline readiness, heat decay, and the steady-state slope. When a configuration's attractor-bucket counts reach characteristic thresholds, L8 transitions

the runtime to a different operating mode, adjusting the balance of loops and their parameters to suit the new workload regime. The SSM passes through four phases: transient (warming up, noisy gradients), quasi-steady (emerging patterns), true steady state (smooth heat surface, stable lookups), and chaotic/disruption (triggered by major workload shift).

Six operating modes are defined:

stable The system has converged to a predictable workload; decay rates and window sizes are held conservatively to avoid disrupting an established optimum.

volatile High workload diversity; the window is widened and inference runs more frequently to track rapidly changing heat surfaces.

diverse Moderate, sustained diversity; window and decay parameters are tuned for broad pattern capture.

temporal Time-structured workloads where periodically recurring patterns dominate; the window is sized to capture at least one full period.

transition The system is moving between attractors; parameters are relaxed to allow rapid re-convergence.

omni All-loops active at maximum sensitivity; intended for initial warm-up or after a major disruption.

The L8 mode selector is not a loop with an accumulation/stabilization pair; it is a higher-order coordinator that reconfigures the parameters of the seven loops based on observed steady-state behavior. The six L8 variants are also available as separate capsule personalities: `capsules/init-l8-{stable`

3.10 Steady-State Machine Theory

The theoretical basis for the adaptive runtime is the Steady-State Machine (SSM), a computational model whose operational state is defined not by discrete symbolic transitions but by convergence to an optimal runtime equilibrium. Using execution statistics as a proxy for thermodynamic quantities, the SSM tracks execution heat H, entropy and diversity measures E, pipeline activation thresholds P, a heat-decay profile Θ, and the sliding execution window W. The governing dynamic is:

$$M_{t+1} = f(M_t,\ \Delta H,\ \Delta W,\ \Delta E), \tag{3.4}$$

where f drives the machine toward a stable basin. The machine is defined by its convergence behavior, not by its instruction sequence.

Five axioms characterize the SSM:

Convergence Every sustained workload induces the SSM to converge toward a stable attractor unless external entropy forces divergence.

Locality of Heat Execution heat reflects both locality and temporal relevance; locality produces stability.

Adaptive Reordering Dictionary reordering is thermodynamic self-optimization, not symbolic mutation.

Stability Maximizes Throughput The steady state is always faster than the cold state.

Perturbation Response When disrupted, the SSM seeks a new steady state; it does not thrash indefinitely.

3.11 Configuration Knobs

Any loop can be disabled at build time. Setting all seven flags to zero yields the baseline configuration used as the DoE reference point.

```
# Disable a single loop
make ENABLE_LOOP_3_LINEAR_DECAY=0

# Baseline build (all loops off)
make ENABLE_LOOP_1_HEAT_TRACKING=0 \
     ENABLE_LOOP_2_ROLLING_WINDOW=0 \
     ENABLE_LOOP_3_LINEAR_DECAY=0 \
     ENABLE_LOOP_4_PIPELINING_METRICS=0 \
     ENABLE_LOOP_5_WINDOW_INFERENCE=0 \
     ENABLE_LOOP_6_DECAY_INFERENCE=0 \
     ENABLE_LOOP_7_ADAPTIVE_HEARTRATE=0
```

Knob	Default	Description
ROLLING_WINDOW_SIZE	4096	Initial/maximum window size
ADAPTIVE_SHRINK_RATE	75	% retained when shrinking
ADAPTIVE_MIN_WINDOW_SIZE	256	Floor for window shrinking
ADAPTIVE_CHECK_FREQUENCY	256	Executions between diversity checks
ADAPTIVE_GROWTH_THRESHOLD	1	Growth rate (%) signalling saturation
DECAY_RATE_PER_US_Q16	1	Heat decay per microsecond ($\mathbb{Q}_{48.16}$)
HEARTBEAT_INFERENCE_FREQUENCY	5000	Ticks between full inference runs
TRANSITION_WINDOW_SIZE	2	Context depth for pipelining prediction

Table 3.2: Principal tuning knobs for the feedback loops.

The implementation is distributed across `src/rolling_window_of_truth.c` (Loop #2), `src/dictionary_heat_optimization.c` (Loops #1 and #3), `src/inference_engine.c` (unified Loops #5 and #6), `src/physics_pipelining_metrics.c` (Loop #4), and the `vm_tick()` heartbeat dispatcher in `src/vm.c` (Loop #7). Knob definitions live in `include/rolling_window_knobs.h`.

3.12 Determinism and Formal Properties

3.13 Hot-Words Cache Optimization

3.14 Hot-Words Cache Performance Analysis

3.14.1 Summary

The StarForth hot-words cache achieves a statistically confirmed **1.78**× speedup for dictionary lookups, with a 95% Bayesian credible interval of $[1.75\times, 1.81\times]$. All measurements use Q48.16 fixed-point arithmetic; no floating-point computation is required.

3.14.2 Experimental Configuration

- **Build:** `make ENABLE_HOTWORDS_CACHE=1 fastest` (x86_64, full assembly optimisations, LTO, direct threading)
- **Benchmark:** 100,000 dictionary lookups via FORTH test harness; word mix: common FORTH primitives (`IF`, `DUP`, `DROP`, `+`, `-`, `@`, `!`, etc.)
- **Cache:** 32 entries; promotion threshold: 50 executions; eviction: LRU

- **Precision:** Q48.16 64-bit fixed-point (nanoseconds)

3.14.3 Lookup Distribution

Path	Count	Fraction
Cache hits	1,415	35.64%
Bucket hits	1,861	46.88%
Misses	694	17.48%
Total	3,970	100%

3.14.4 Latency Results

Path	Min (ns)	Mean (ns)	Max (ns)
Cache path (1,415 samples)	22	31.543	101
Bucket path (1,861 samples)	23	56.237	370

3.14.5 Speedup Analysis

$$\text{speedup} = \frac{\bar{t}_{\text{bucket}}}{\bar{t}_{\text{cache}}} = \frac{56.237\,\text{ns}}{31.543\,\text{ns}} = 1.78\times \tag{3.5}$$

Time saved per cache hit: $56.237 - 31.543 = 24.694$ ns (43.9% reduction).

Bayesian credible intervals.

Interval	Speedup
95% credible	$[1.75\times,\ 1.81\times]$
99% credible	$[1.73\times,\ 1.83\times]$

$P(\text{speedup} > 1.1\times) = 99.9\%$. All credible-interval arithmetic is performed in pure Q48.16 integer arithmetic; no floating-point or libm functions are used.

3.14.6 Cache Management Behaviour

The physics engine promoted 10 words automatically from the dictionary into the cache based on execution entropy exceeding the threshold of 50. No manual tuning was applied. Zero evictions occurred during the benchmark; the 32-entry cache was sufficient for the active working set.

The two highest-entropy words at experiment completion were `EXIT` (entropy 114) and `LIT` (entropy 101), consistent with their frequency in the compiled FORTH test harness.

3.14.7 Production Properties

- **No floating-point:** All arithmetic is Q48.16 integer; the cache subsystem is compatible with L4Re and bare-metal targets.
- **No external libraries:** No libm dependency.
- **Deterministic:** Near-zero variance across 3,970 samples confirms formally proven deterministic behaviour.
- **Linear complexity:** Cache lookup is O(1); bucket search is linear in bucket depth.

3.14.8 Test Suite Validation

```
make fastest ENABLE_HOTWORDS_CACHE=1
make test
# Total: 782  Passed: 731  Failed: 0  Skipped: 49  Errors: 0
```

All 731 implemented tests pass with the cache-enabled build.

3.15 Optimization Proposals

3.16 Physics-Driven Optimisation Proposals

3.16.1 Strategic Direction

The hot-words cache experiment (1.78×, 95% CI: [1.75×, 1.81×]) validates the physics-driven optimisation methodology: collect execution-frequency metrics at runtime, make automatic promotion decisions, and measure impact with statistical rigour. Nine additional opportunities build on the same foundation.

JIT compilation is explicitly excluded from consideration. JIT requires runtime code generation, violates L4Re microkernel policy, introduces floating-point overhead, and defeats formal verification. Physics-driven optimisation achieves measurable gains within StarForth's verification and portability constraints.

3.16.2 Opportunity Catalogue

Opportunity 1 — Return Stack Prediction and Colon Word Inlining. Frequently called colon definitions with small bodies (fewer than 256 bytes) are candidates for compile-time inlining when `execution_heat > 100`. Inlining eliminates dictionary lookup and return-stack overhead. Expected gain: 1.3–2.0× for call-heavy workloads. Verification: static (inlining correctness is provable via Isabelle/HOL).

Opportunity 2 — Block I/O Prefetching. A Markov-style transition matrix tracks which block LBN follows which. When a block is accessed and the next-block heat exceeds a threshold, that block is pre-fetched into the buffer cache. Expected gain: 1.5–3.0× for block-sequential access patterns. Implementation: medium complexity.

Opportunity 3 — Stack Operation Fusion. Co-execution heat tracks consecutive word pairs (e.g., `DUP DROP`, `SWAP ROT`, `OVER SWAP`). Pairs exceeding a heat threshold at compile time are fused into dedicated primitives, eliminating two lookups and one execution per pair. Expected gain: 1.2–1.5× for stack-heavy programs. Verification: static (composition of verified primitives).

Opportunity 4 — Memory Allocation Pattern Prediction. Allocation frequency by size is tracked in a heat vector. Sizes exceeding a threshold trigger pre-warmed pool maintenance. Repeating allocation sequences are detected and cached. Expected gain: 1.3–1.8× for allocation-heavy programs.

Opportunity 5 — Control Flow Branch Prediction. `IF`/`THEN`/`ELSE` outcomes are tracked per source location. Branches with greater than 90% one-sided bias are annotated with a prediction hint that the inner interpreter can use to reorder code or emit x86 branch-hint prefixes. Expected gain: 1.1–1.3× (architecture-dependent).

Opportunity 6 — Vocabulary Search Path Reordering. Per-vocabulary hit rates are tracked. The search order is sorted dynamically by hit rate, placing the most productive vocabulary first. Expected gain: 1.2–1.6× in multi-vocabulary programs. Implementation: low complexity.

Opportunity 7 — String Operation Batching. Consecutive string-output operations (`."`, `EMIT`, `TYPE`) detected at compile time are fused into a single batch output, reducing interpreter loop iterations. Expected gain: 1.1–1.4× for I/O-bound programs. Implementation: low complexity.

Opportunity 8 — Arithmetic Operation Reordering. For hot commutative operations at a given source location, operand sizes are compared and the smaller operand is loaded first to improve prefetch behaviour. Expected gain: 1.05–1.15× (architecture-dependent).

Opportunity 9 — Word Placement Optimisation. Co-execution heat between word pairs guides memory compaction: frequently co-executed words are relocated adjacent to each other in the dictionary to improve instruction-cache locality. Expected gain: 1.05–1.2×. Implementation: high complexity (requires GC integration).

3.16.3 Implementation Roadmap

Phase	Opportunities	Rationale
1 (complete)	Hot-words cache	Proven; production-ready
2 (recommended next)	#3, #6, #1	Highest ROI, lowest complexity
3 (future)	#2, #7, #4	Medium complexity
4 (advanced)	#5, #8, #9	CPU-specific or GC-dependent

3.16.4 Expected Cumulative Gains

Conservative sequential composition of Phases 1–2:

$$1.78\times \;\times 1.2 \;\times\; 1.2 \;\times\; 1.3 \;\approx\; 3.3\times \text{ cumulative speedup}$$

All nine opportunities together: 5–8× total improvement is plausible, subject to workload dependency and diminishing returns.

3.16.5 Unified Metrics Infrastructure

All nine opportunities require co-execution heat tracking and sequence pattern buffers not yet present in the VM. A one-time extension to the `PhysicsMetrics` per-word structure adds: a co-execution heat vector, timing accumulator, per-word cache-line counters, and a small recent-execution circular buffer. This single investment unlocks the infrastructure required for Opportunities 1–9.

Chapter 4

Formal Verification

StarForth's physics-driven adaptive runtime and FORTH-79 word set are formally verified using Isabelle/HOL. Unlike assertions or unit tests, Isabelle/HOL produces machine-checked proofs: every theorem is mechanically verified from first principles within a small, well-audited logical kernel, providing a level of assurance that runtime testing alone cannot reach. The proof corpus consists of 24 theory files covering all seven physics feedback loops, six FORTH-79 word categories, the Q48.16 fixed-point arithmetic subsystem, concurrency and mutual exclusion properties, and the word-level ACL security system.

4.1 Proof Infrastructure

All theory files reside in the `proof/` directory at the repository root. The session is declared in `proof/ROOT`, which lists the import order and the session name for the Isabelle build system. All 24 theory files import either `StarForth_Base` or a theory that already imports it, ensuring that the base VM state specification serves as the unique ground truth for all downstream proofs.

To check all proofs:

```
isabelle build -D proof/
```

The proof status convention used throughout the corpus distinguishes three levels: a check mark (✓) denotes a fully mechanised result with no outstanding human-audit obligation; a circle (○) denotes a proof that depends on an explicit named axiom, where the corresponding C code must be manually verified to satisfy the axiom; and a warning symbol (△) denotes a use of `sorry` or a placeholder that must be resolved before the proof is considered complete. No file in the current corpus uses `sorry`.

4.2 Base Definitions

`StarForth_Base.thy` is the ground-truth specification of the StarForth VM state. It defines the `vm_state` record type with all fields corresponding one-to-one to `include/vm.h`: data stack, return stack, memory, dictionary, physics substate (rolling window, heat, decay slope, pipeline metrics, heartbeat), and lock fields. It defines the fundamental predicates `wf_vm` (well-formedness), `ds_full` (data stack at capacity), and `rs_full` (return stack at capacity), which must be audited against the corresponding C guard expressions in `vm.c` and `word_source/`.

The authority protocol is explicit: C code is written to match this theory, not the other way around. Discrepancies between this theory and the C source are treated as bugs in the C code. Every field and predicate carries a `CODE-MUST-MATCH` annotation identifying the corresponding C declaration. `StarForth_Base.thy` also imports `Main` from the Isabelle/HOL standard library and exports definitions to all downstream theories.

4.3 Physics Loop Proofs

Seven theory files establish the formal properties of the physics feedback loops described in Chapter 3.

4.3.1 Loop #1: Execution Heat

Theory: `StarForth_Loop1_Heat.thy`. Mirrors: `src/dictionary_heat_optimization.c`.

The theory proves five properties of the heat lifecycle: heat is non-negative throughout its lifecycle; heat increment strictly increases `de_heat`; heat is bounded above by a `MAX_HEAT` constant (preventing integer overflow in the C field); `WORD_FROZEN` words are immune to decay steps (their heat field is not modified by the decay sweep); and `WORD_PINNED` words hold a heat value of at least 1 after any decay step. All five results are fully mechanised.

4.3.2 Loop #2: Rolling Window Bounded Growth

Theory: `StarForth_Loop2_Window.thy`. Mirrors: `src/rolling_window_of_truth.c`.

The theory models two window-size quantities: `rw_eff_window` (the effective window size, which is adaptive and bounded to [`ADAPTIVE_MIN`, `ROLLING_WINDOW_SIZE`]) and `rw_act_window` (the actual fill level of the ring buffer, which grows monotonically until it saturates at `ROLLING_WINDOW_SIZE`). The central result establishes that the effective window size can never escape its bounds regardless of how many shrink or grow operations the inference engine applies. This prevents the window from collapsing to zero or exceeding the buffer capacity.

4.3.3 Loop #3: Decay Convergence

Theory: `StarForth_Loop3_Decay.thy`. Imports: `StarForth_Q48_16`, `StarForth_Base`. Mirrors: `src/vm_time.c`, `include/vm.h`.

The theory proves that the decay operation is non-negative (heat after decay is $\max(0, \ldots)$, so it cannot go negative), monotonically decreasing (repeated decay sweeps never increase heat), and that `WORD_FROZEN` words are fully immune to decay steps. The decay constant `DECAY_RATE_PER_US_Q16` and the adaptive slope `decay_slope_q48` are both modelled as $\mathbb{Q}_{48.16}$ quantities using the fixed-point theory.

4.3.4 Loop #4: Pipelining Metrics Safety

Theory: `StarForth_Loop4_Pipeline.thy`. Imports: `StarForth_Q48_16`, `StarForth_Base`. Mirrors: `src/physics_pipelining_metrics.c`.

The theory establishes safety properties for the transition-probability computation and prefetch machinery: transition counts are non-negative; the probability formula produces a result in $[0, 1]$ (in $\mathbb{Q}_{48.16}$ fixed point); and prefetch-hit/miss accounting preserves the invariant that `prefetch_hits` never exceeds `prefetch_attempts`.

4.3.5 Loop #5: Window Width Inference Termination

Theory: `StarForth_Loop5_WinInf.thy`. Imports: `StarForth_Q48_16`, `StarForth_Loop2_Window`. Mirrors: `src/inference_engine.c`.

The theory proves that the binary-chop window suggestion always produces a value within [`ADAPTIVE_MIN_WINDOW_SIZE`, `ROLLING_WINDOW_SIZE`] before the result is written to the VM, so the effective window size invariant from Loop #2 is preserved by inference updates. The statistical correctness of the Levene F-test is taken as an explicit named axiom (`inference_window_clamped_valid`), carrying a documented audit obligation against `run_inference()` in `src/inference_engine.c`. This is the only axiom in the corpus that depends on a mathematical result external to the HOL standard library; it is documented as such and does not use `sorry`.

4.3.6 Loop #6: Decay Slope Inference Soundness

Theory: `StarForth_Loop6_DecayInf.thy`. Imports: `StarForth_Q48_16`, `StarForth_Loop3_Decay`. Mirrors: `src/inference_engine.c`.

The theory proves that the OLS (ordinary least-squares) regression result is always clamped to [`DECAY_SLOPE_MIN`, `DECAY_SLOPE_MAX`] before being written to the VM state, so the decay slope remains well-formed regardless of the numerical output of the regression. Unlike Loop #5, no statistical axiom is required; all invariants follow from the clamping alone. The theory also proves that no update occurs when `io_early_exited` is set (the ANOVA early-exit guard).

4.3.7 Loop #7: Heartrate Adaptive Stability

Theory: `StarForth_Loop7_Heartrate.thy`. Mirrors: `src/vm_time.c`, `include/vm.h`.

The theory proves that `tick_target_ns` is always strictly greater than zero, remains within [`TICK_MIN_NS`, `TICK_MAX_NS`] under all adjustment operations (both lengthening and shortening the period), and that `hb_tick_count` increases monotonically—meaning the tick counter never wraps backward or resets unexpectedly during normal operation.

4.4 Word-Category Proofs

Six theory files verify the FORTH-79 word implementations against the base VM state model.

Theory file	Word category	Principal result
`StarForth_Stack_Words.thy`	Stack manipulation (`DUP`, `DROP`, `SWAP`, `ROT`, ...)	Stack-depth arithmeti
`StarForth_Arithmetic_Words.thy`	Arithmetic (`+`, `-`, `*`, `/`, `MOD`, ...)	Binary-op lifting; HOI
`StarForth_Logical_Words.thy`	Logic and comparison (`AND`, `OR`, `XOR`, `INVERT`, ...)	FORTH-79 boolean co
`StarForth_Return_Stack_Words.thy`	Return stack (`>R`, `R>`, `R@`, `RDROP`, ...)	Bidirectional stack tra
`StarForth_Memory_Words.thy`	Memory (`@`, `!`, `C@`, `C!`, `MOVE`, `FILL`)	Read-after-write corre
`StarForth_Q48_16.thy`	Q48.16 fixed-point arithmetic	HOL-Word 64-bit mo

Table 4.1: Word-category proof inventory.

All word-category theories import `StarForth_Base` and use HOL integers (arbitrary precision) as the semantic domain, with the understanding that proofs hold in C provided no intermediate value overflows 64-bit signed range. `StarForth_Q48_16.thy` uses `HOL-Library.Word` to model the 64-bit fixed-point type with the same wrapping arithmetic as C `uint64_t`, closing the gap that an earlier natural-number model left open.

4.5 Concurrency and Mutual Exclusion

Three theory files establish the concurrency model and its safety properties.

`StarForth_Transition.thy` (imports `StarForth_Mutex`) defines the Labeled Transition System governing the interaction between the execution thread (`ExecThread`) and the heartbeat thread (`HeartbeatThread`). It introduces the quotient relation `exec_equiv` ($\simeq$) and establishes two explicit named axioms: `heartbeat_exec_neutral` (the heartbeat step is the identity in `vm_state`/$\simeq$; audited against `src/vm_time.c`) and `word_physics_transparent` (word execution is a congruence law for $\simeq$; audited against the Level-1 word theories). The former eight field-level axioms present in earlier versions of the theory are now proved from these two axioms via the `exec_equiv` quotient.

`StarForth_Mutex.thy` (imports `StarForth_Base`) models the two VM locks—`tuning_lock` (guards physics-loop parameter updates) and `dict_lock` (guards dictionary writes)—as first-class `vm_state` fields of type `lock_state` (`LockFree` | `LockHeld nat`). The mutual-exclusion

safety property is proved: at most one thread holds a given lock at any time. Since StarForth has exactly two threads (execution and heartbeat), thread identifiers are 0 (`ExecThread`) and 1 (`HeartbeatThread`).

`StarForth_Concurrent.thy` (imports `StarForth_Transition`, `StarForth_Loop7_Heartrate`, `StarForth_Loop3_Decay`) proves non-interference: the data-stack result of executing any word is the same regardless of how many heartbeat steps have interleaved, formalized as the `heartbeat_noninterference` theorem:

```
theorem heartbeat_noninterference:
  "\<forall>n␣k␣vm.
␣␣␣␣␣data_stack␣(word_table␣(heartbeat_step␣^^␣k␣$␣vm)␣n
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣(heartbeat_step␣^^␣k␣$␣vm))
␣␣␣␣␣=␣data_stack␣(word_table␣vm␣n␣vm)"
```

This result, which is sorry-free and proved from the two axioms in `StarForth_Transition`, establishes that arbitrary word sequences are independent of heartbeat timing—the adaptive physics machinery does not disturb the observable stack behavior of executing programs.

4.6 Overall Correctness

`StarForth_Correctness.thy` is the top-level theory that imports all twelve subordinate theories and assembles the totality result. It is sorry-free, with an explicit inventory of two axioms (both inherited from `StarForth_Transition`) and no others. The top-level correctness theorem binds the word-category correctness results, the loop invariants, and the concurrency non-interference result into a single statement asserting that the StarForth VM satisfies its specification across all defined operating conditions.

4.7 Proof Inventory

Table 4.2 lists all 24 theory files with a one-line description of the principal result or role of each.

4.8 ACL System Proofs

Five theory files verify the word-level ACL system implemented in `src/word_source/acl_words.c` and `capsules/ACL.4th`. All five import `StarForth_Base` or a theory that imports it; all are sorry-free.

`ACL_Pin_Monotone.thy` proves that the `acl_pinned` field in `DictEntry` is a set-only (monotone) field: once written to 1 by `ACL-PIN`, no subsequent operation—`ACL-MODE!`, `ACL-TTL!`, or `ACL-ALLOW!`—can clear it to 0. The `CODE-MUST-MATCH` annotation lists four C functions that must never write `acl_pinned=0` after it has been set. This property ensures that pinned words retain their immutability guarantee for the lifetime of the VM session.

`ACL_Inherit_Clears_Pin.thy` proves that `ACL-INHERIT` always clears `acl_pinned` on the *destination* entry, even when the source entry is pinned. The `acl_inherit_entry()` C function mirrors this by setting `dst->acl_pinned = 0` unconditionally. This prevents inherited ACL from accidentally propagating the immutability of a privileged parent word to a child.

`ACL_TTL_Bounded.thy` proves three properties of the adaptive TTL mechanism: the output of `ACL-TTL-COMPUTE` is always at least `ACL_BASE_TTL` (minimum service guarantee); it is always at most `ACL_MAX_TTL` (bounded cost amortization); and the hot-path TTL decrement is safe (TTL cannot go below 0 when the pre-condition holds). Additionally, STRICT mode holds TTL at 0 after every `ACL-RECHECK`.

`ACL_Emergency_Bypass.thy` proves that when either `emergency_console=1` (fault handler active) or `zuse_session=1` (superuser authenticated) is set in the VM flags, all ACL checks

Theory file	Principal result or role
StarForth_Base.thy	Ground-truth VM state type; base predicates (wf_vm, etc.)
StarForth_Q48_16.thy	Q48.16 fixed-point: HOL-Word 64-bit model matching C uint64_t
StarForth_Stack_Words.thy	Stack-manipulation words: depth arithmetic, underflow error
StarForth_Arithmetic_Words.thy	Arithmetic words: binary-op lifting, HOL-int semantics
StarForth_Logical_Words.thy	Logical/comparison words: FORTH-79 boolean convention
StarForth_Return_Stack_Words.thy	Return-stack words: bidirectional transfer, capacity bounds
StarForth_Memory_Words.thy	Memory words: read-after-write correctness, no spurious mutation
StarForth_Loop1_Heat.thy	Loop #1: heat non-negativity, MAX_HEAT bound, FROZEN/PINNED s
StarForth_Loop2_Window.thy	Loop #2: effective window bounded in [ADAPTIVE_MIN, ROLLING_W
StarForth_Loop3_Decay.thy	Loop #3: decay non-negativity, monotone decrease, FROZEN immunity
StarForth_Loop4_Pipeline.thy	Loop #4: transition-probability in $[0, 1]$, hit/attempt accounting safety
StarForth_Loop5_WinInf.thy	Loop #5: inference output clamped in bounds; Levene axiom audited
StarForth_Loop6_DecayInf.thy	Loop #6: OLS slope clamped, no update on early-exit (sorry-free)
StarForth_Loop7_Heartrate.thy	Loop #7: tick period in [TICK_MIN_NS, TICK_MAX_NS], monotone count
StarForth_Mutex.thy	Mutual exclusion: at most one thread holds tuning_lock/dict_lock
StarForth_Transition.thy	Labeled transition system; exec_equiv quotient; 2 explicit axioms
StarForth_Concurrent.thy	Non-interference: heartbeat steps do not affect observable stack results
StarForth_Correctness.thy	Top-level totality theorem; imports all 12 theories; sorry-free
ACL_Pin_Monotone.thy	acl_pinned is set-only: once set to 1, never cleared to 0
ACL_Inherit_Clears_Pin.thy	ACL-INHERIT always clears acl_pinned on the destination entry
ACL_TTL_Bounded.thy	TTL output is $\geq$ ACL_BASE_TTL and $\leq$ ACL_MAX_TTL; safe decrement
ACL_Emergency_Bypass.thy	emergency_console=1 or zuse_session=1 bypasses all ACL checks
ACL_No_Escalation.thy	A child word cannot acquire acl_pinned=1 via ACL-INHERIT alone
ROOT	Isabelle session manifest (not a theory; declares import order)

Table 4.2: All 24 entries in the proof/ corpus (23 .thy files + ROOT).

are bypassed. The two conditions are independent: the emergency console flag is set only by C code (acl_recheck, sk_repl) during error recovery; the superuser flag is set only by the ZUSE-AUTHENTICATE C primitive. The theory proves that neither condition can be established from within the ACL policy layer itself.

ACL_No_Escalation.thy (imports ACL_Inherit_Clears_Pin, ACL_TTL_Bounded, ACL_Emergency_Bypa
proves the central security property: a child word cannot acquire acl_pinned=1 through ACL-INHERIT alone. Inheriting from any source word—pinned or not—always leaves the destination un-pinned. A word can become pinned only through an explicit ACL-PIN call, which requires intentional action from a trusted caller. This prevents capsule words from upgrading themselves to kernel-level immutability by inheriting from a privileged parent.

Chapter 5

Experimental Results

5.1 Experimental Design

5.2 Primary Result: Steady-State Convergence

5.3 Claim Table

5.4 Negative Results and Limitations

5.5 Sensitivity Analysis

5.6 Reproducing the Results

Chapter 6

Build and Deployment

6.1 Prerequisites

6.2 Build Targets

Listing 6.1: StarForth build targets

```
make                # Standard optimized build (auto-detects
    architecture)
make fastest        # Maximum performance (ASM + LTO + direct
    threading)
make pgo            # Profile-guided optimization
make debug          # Debug build with symbols
make test           # Full test suite (936+ tests)
make bench          # Quick benchmark
make clean          # Remove build artifacts
```

6.3 Build Flags

6.4 Running

Listing 6.2: Running StarForth

```
./build/amd64/standard/starforth              # Interactive REPL
./build/amd64/standard/starforth --run-tests  # Tests then REPL
./build/amd64/standard/starforth -c "1␣2␣+␣.␣BYE"  # Inline code
./build/amd64/fastest/starforth --doe         # DoE mode
```

6.5 Test Suite

Chapter 7

Word-Level ACL System

7.1 Design Summary

The word-level ACL system is implemented through Phase 6. Key constraints (cite the design doc for full details):

- All policy logic in `ACL.4th` — no new C primitives for policy.
- Four `DictEntry` C fields: `acl_ttl`, `acl_allow`, `acl_mode`, `acl_pinned`.
- Two VM flags: `emergency_console` (fault handler active) and `zuse_session` (superuser authenticated).
- `ACL.4th` is self-activating; `init.4th` only needs `S" ACL.4th" EXEC`.
- `ACL-PIN` is one-way; inheritance clears pin, copies mode.

7.2 Phase Status

Table 7.1: ACL implementation phase status

Phase	Description	Status
1	C Infrastructure (`DictEntry` fields, interpreter hook, `acl_recheck()`)	Done
2	`ACL.4th` FORTH policy words + `ACL-INIT-PRIMITIVES` + self-activation	Done
3	`capsules/zuse.4th` bootstrap superuser skeleton; CA root placeholder	Done
4	`init.4th` opt-in toggle	Done
5	POST tests (800/800) + Isabelle/HOL proofs (5 theory files)	Done
6	`EMERGENCY_CONSOLE_ENABLED` build flag + `vm_fault_handler`	Done
7	LithosAnanke parity — port to kernel context, three-arch acceptance	Pending
8	PKI / thumbdrive — Ed25519 challenge-response; user minting by zuse	Future

7.3 ACL Formal Proofs

Five Isabelle/HOL theorems cover the ACL system's core safety properties:

1. **Pin Monotonicity** (`ACL_Pin_Monotone.thy`) — once pinned, a word remains pinned through the system's lifetime.

2. **Inheritance Clears Pin** (`ACL_Inherit_Clears_Pin.thy`) — inherited words carry mode but never carry pin status.

3. **TTL Bounded** (`ACL_TTL_Bounded.thy`) — all TTL values remain within defined bounds; no overflow.

4. **Emergency Bypass Correct** (`ACL_Emergency_Bypass.thy`) — the emergency console bypass does not escalate privileges.

5. **No Escalation** (`ACL_No_Escalation.thy`) — no sequence of ACL operations allows a word to acquire permissions it was not granted.